Case Study 1: Progressive Electricity Billing

1. Problem Statement

Develop a Python program to calculate household electricity bills based on the number of units consumed. The program should apply different electricity rates for different consumption slabs, add fixed charges, apply surcharge or rebate conditions, compare monthly bills and identify unusual increases.

2. Objectives

- Calculate electricity bills using progressive slabs.
- Add fixed charges.
- Apply surcharge and rebate conditions.
- Compare electricity bills across months.
- Identify unusual increases in electricity bills.

3. Assumptions

The following sample billing rules are assumed because the PDF does not specify exact rates.

Electricity units	Rate
First 100 units	₹2 per unit
101–200 units	₹3 per unit
Above 200 units	₹5 per unit

Additional rules:

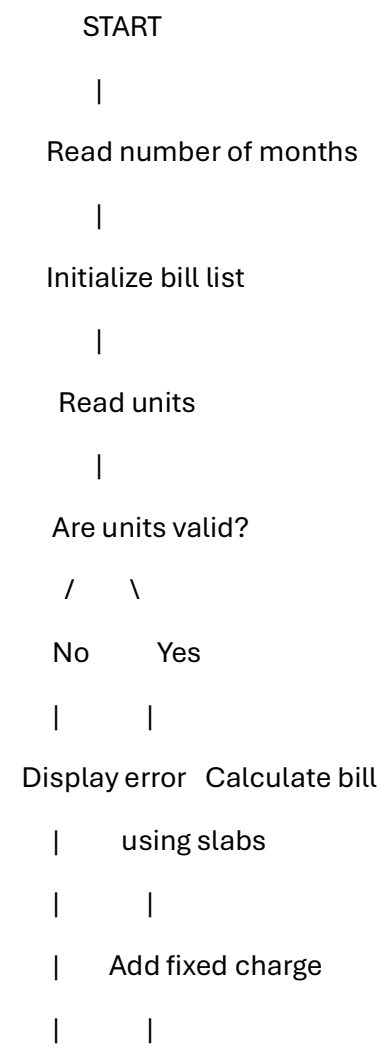
- Fixed charge = ₹50.
- Surcharge = 10% if the bill exceeds ₹1,000.
- Rebate = ₹50 if consumption is below 50 units.
- An increase above ₹100 compared with the previous month is considered unusual.

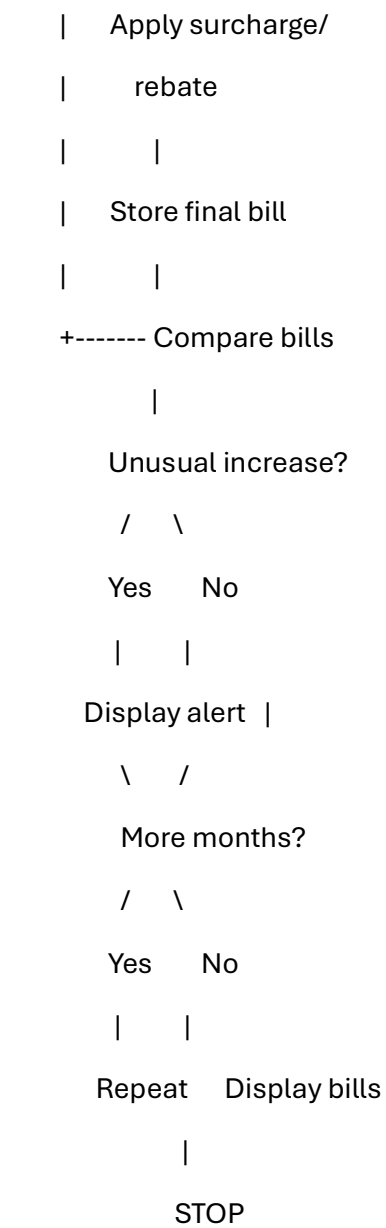
4. Algorithm

1. Start.
2. Initialize an empty list to store monthly bills.
3. Read the number of months.

4. For each month, read electricity units.
5. Check whether the units are valid.
6. Calculate the bill according to the electricity slabs.
7. Add fixed charges.
8. Apply surcharge or rebate.
9. Store the final bill.
10. Compare each month's bill with the previous month's bill.
11. Display unusual increases.
12. Stop.

5. Flowchart





6. Pseudocode

START

INPUT number_of_months

CREATE empty bills list

FOR each month

```
INPUT units

IF units < 0
   DISPLAY Invalid units

ELSE
   IF units <= 100
      bill = units * 2

   ELSE IF units <= 200
      bill = 100 * 2 + (units - 100) * 3

   ELSE
      bill = 100 * 2 + 100 * 3
           + (units - 200) * 5

   END IF

   bill = bill + 50

   IF bill > 1000
      bill = bill + 10% surcharge

   ELSE IF units < 50
      bill = bill - 50
   END IF

   STORE bill
```

END IF

END FOR

COMPARE monthly bills

DISPLAY unusual increases

STOP

7. Python Program

```
months = int(input("Enter number of months: "))

bills = []

for i in range(months):

    units = int(input("Enter electricity units: "))

    if units < 0:
        print("Invalid units")
        bills.append(None)
        continue

    if units <= 100:
        bill = units * 2

    elif units <= 200:
```

```
        bill = 100 * 2 + (units - 100) * 3

    else:
        bill = 100 * 2 + 100 * 3 + (units - 200) * 5

    bill += 50

    if bill > 1000:
        bill += bill * 0.10

    elif units < 50:
        bill -= 50

    bills.append(round(bill, 2))

    print("Monthly Bill: Rs.", round(bill, 2))

print("\nMONTHLY BILL COMPARISON")

for i in range(len(bills)):

    if bills[i] is None:
        continue

    print("Month", i + 1, ":", bills[i])

    if i > 0 and bills[i - 1] is not None:
```

```
    increase = bills[i] - bills[i - 1]


    if increase > 100:
        print("Unusual increase detected!")
```

8. Test Cases

Test	Units	Expected result
1	50	₹150
2	100	₹250
3	150	₹415
4	250	₹885
5	-10	Invalid units

9. Conclusion

The Progressive Electricity Billing program calculates electricity bills using different consumption slabs. It also applies fixed charges, surcharges and rebates. Monthly comparison helps identify unusual increases in electricity consumption costs.

Case Study 2: Retail Billing with Multiple Discount Rules

1. Problem Statement

Develop a Python program to process multiple products purchased by a customer. Calculate the price based on quantity, apply discounts according to purchase amounts, include category-specific taxes and generate an itemized bill.

2. Objectives

- Accept multiple product details.
- Calculate product-wise amounts.
- Apply discount rules.
- Calculate category-specific taxes.

- Generate the final customer bill.

3. Assumptions

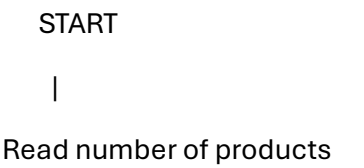
For this program, the following sample rules are assumed:

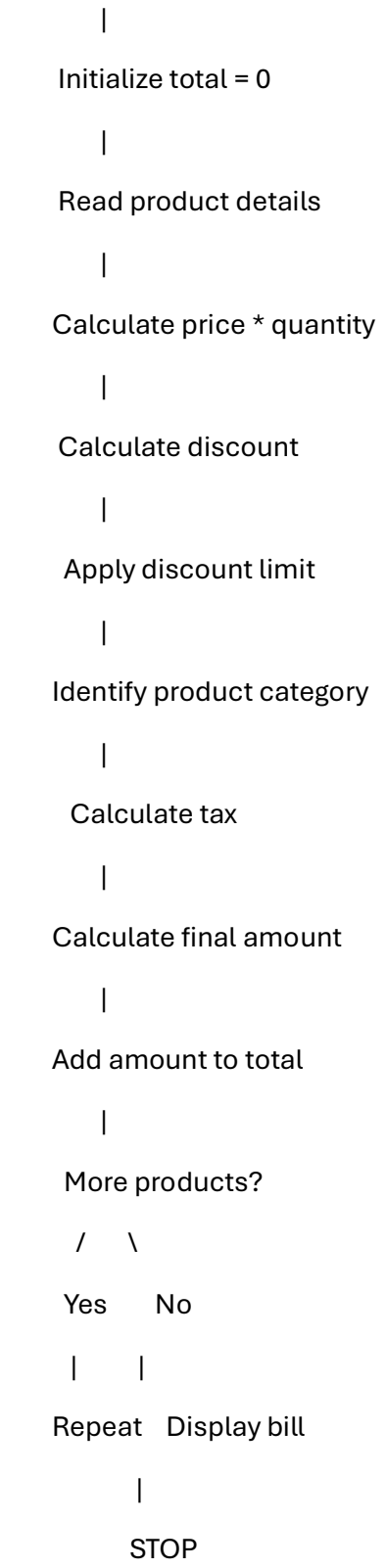
- Discount of 10% for product amounts of ₹1,000 or more.
- Discount of 5% for product amounts between ₹500 and ₹999.99.
- Maximum discount is ₹200 per product.
- Electronics tax = 12%.
- Food tax = 5%.
- Other products tax = 8%.

4. Algorithm

1. Start.
2. Read the number of products.
3. Initialize total bill to zero.
4. Read product name, category, price and quantity.
5. Calculate the product amount.
6. Apply the discount according to the amount.
7. Check the product category.
8. Calculate the applicable tax.
9. Calculate the final product amount.
10. Add it to the total bill.
11. Display the itemized bill.
12. Display the total bill.
13. Stop.

5. Flowchart





6. Pseudocode

START

```
INPUT number_of_products
total = 0

FOR each product
  INPUT name, category, price, quantity

  amount = price * quantity

  IF amount >= 1000
    discount = amount * 10 / 100

  ELSE IF amount >= 500
    discount = amount * 5 / 100

  ELSE
    discount = 0

  END IF

  IF discount > 200
    discount = 200
  END IF

  taxable_amount = amount - discount

  IF category = Electronics
```

```
    tax = taxable_amount * 12 / 100

  ELSE IF category = Food
    tax = taxable_amount * 5 / 100

  ELSE
    tax = taxable_amount * 8 / 100

  END IF

  final_amount = taxable_amount + tax

  total = total + final_amount

  DISPLAY product bill

END FOR

DISPLAY total

STOP
```

7. Python Program

```
n = int(input("Enter number of products: "))

total = 0

for i in range(n):
```

```
name = input("\nEnter product name: ")
category = input("Enter category: ").lower()

price = float(input("Enter price: "))
quantity = int(input("Enter quantity: "))

if price < 0 or quantity <= 0:
    print("Invalid product details")
    continue

amount = price * quantity

if amount >= 1000:
    discount = amount * 0.10

elif amount >= 500:
    discount = amount * 0.05

else:
    discount = 0

discount = min(discount, 200)

taxable_amount = amount - discount

if category == "electronics":
    tax = taxable_amount * 0.12
```

```
    elif category == "food":
        tax = taxable_amount * 0.05

    else:
        tax = taxable_amount * 0.08

    final_amount = taxable_amount + tax

    total += final_amount

    print("\nProduct:", name)
    print("Amount:", round(amount, 2))
    print("Discount:", round(discount, 2))
    print("Tax:", round(tax, 2))
    print("Final Amount:", round(final_amount, 2))

print("\nTOTAL BILL: Rs.", round(total, 2))
```

8. Test Cases

Product	Category	Price	Quantity	Final amount
Laptop	Electronics	10,000	1	₹11,096
Rice	Food	600	1	₹598.50
Pen	Other	100	2	₹216

9. Conclusion

The Retail Billing program calculates the total cost of multiple products. It applies different discounts and category-based taxes to generate an accurate itemized bill.

Case Study 3: Attendance Recovery Computation

1. Problem Statement

Develop a Python program to calculate a student's current attendance percentage and determine the minimum number of consecutive classes the student must attend to reach the required attendance percentage of 75%.

2. Objectives

- Calculate current attendance percentage.
- Check attendance eligibility.
- Determine additional classes required.
- Display the final attendance percentage.
- Help students understand attendance requirements.

3. Assumptions

- Minimum required attendance = 75%.
- All future classes are attended by the student.
- Attendance must not exceed total classes.
- Total classes must be greater than zero.
- Absent classes are not considered in future calculations.

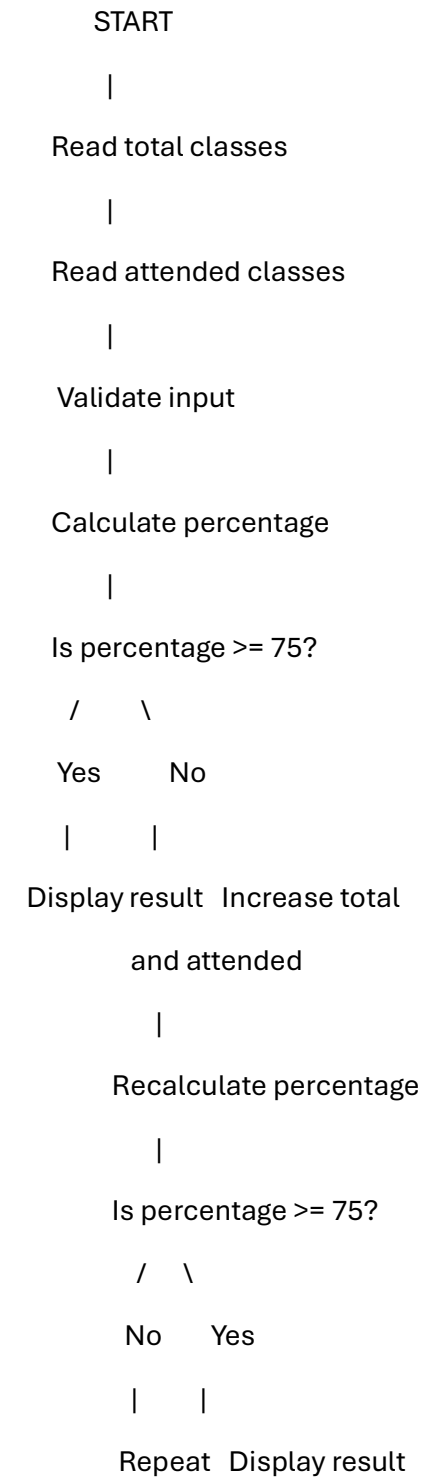
4. Algorithm

1. Start.
2. Read total number of classes.
3. Read attended classes.
4. Validate the input values.
5. Calculate attendance percentage.
6. Check whether attendance is at least 75%.
7. If attendance is below 75%, increase both total classes and attended classes by one.
8. Repeat until attendance reaches 75%.
9. Display additional classes required.

10. Display final attendance percentage.

11. Stop.

5. Flowchart



|

STOP

6. Pseudocode

```
START

INPUT total_classes
INPUT attended_classes

IF total_classes <= 0 OR
  attended_classes < 0 OR
  attended_classes > total_classes

   DISPLAY Invalid attendance

ELSE

   percentage = (attended_classes /
            total_classes) * 100

   extra_classes = 0

   WHILE percentage < 75

      total_classes = total_classes + 1

      attended_classes = attended_classes + 1

      extra_classes = extra_classes + 1
```

```
    percentage = (attended_classes /
                total_classes) * 100

  END WHILE

  DISPLAY extra_classes
  DISPLAY percentage

END IF

STOP
```

7. Python Program

```
total_classes = int(input("Enter total classes: "))
attended_classes = int(input("Enter attended classes: "))

if (total_classes <= 0 or attended_classes < 0
    or attended_classes > total_classes):

  print("Invalid attendance details")

else:

  percentage = (attended_classes / total_classes) * 100

  print("Current Attendance:",
      round(percentage, 2), "%")
```

```
extra_classes = 0

while percentage < 75:

    total_classes += 1
    attended_classes += 1
    extra_classes += 1

    percentage = (attended_classes / total_classes) * 100

print("Additional Classes Required:", extra_classes)

print("Final Attendance:",
      round(percentage, 2), "%")

print("Attendance Eligibility Achieved")
```

8. Test Cases

Test	Total classes	Attended	Expected additional classes
1	100	60	60
2	100	70	20
3	100	75	0
4	50	30	30
5	0	0	Invalid input

9. Sample Output

Enter total classes: 100

Enter attended classes: 60

Current Attendance: 60.0 %

Additional Classes Required: 60

Final Attendance: 75.0 %

Attendance Eligibility Achieved

10. Conclusion

The Attendance Recovery program calculates the current attendance percentage and determines how many additional consecutive classes a student must attend to achieve the required 75% attendance.

Case Study 4: ATM Transaction Sequence Simulation

1. Problem Statement

Develop a Python program to simulate ATM transactions. The program should process deposits and withdrawals, maintain the account balance, enforce withdrawal limits, reject invalid transactions and display a transaction summary.

2. Objectives

- Perform deposit and withdrawal operations.
- Maintain the available account balance.
- Check withdrawal limits.
- Reject invalid transactions.
- Display transaction history.

3. Assumptions

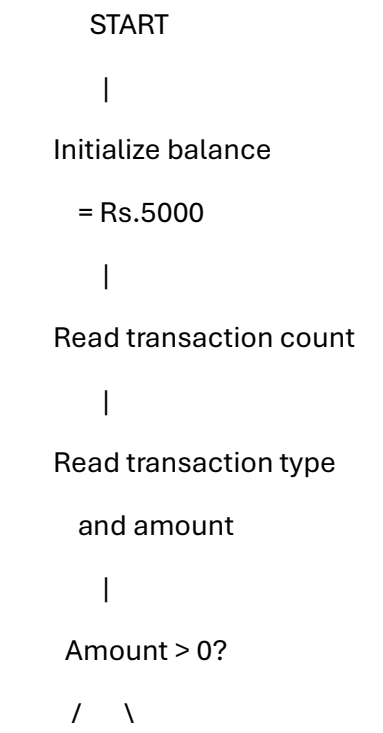
1. Initial bank balance is ₹5,000.
2. Maximum withdrawal per transaction is ₹10,000.
3. Transaction amount must be greater than zero.
4. Withdrawal cannot exceed the available balance.

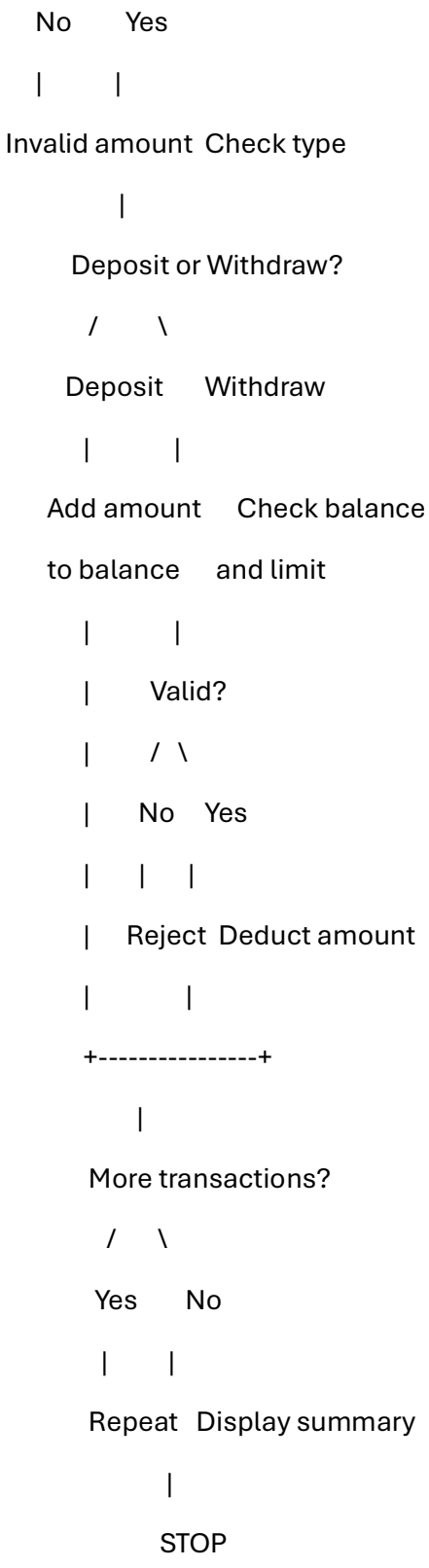
5. Only deposit and withdrawal operations are allowed.

4. Algorithm

1. Start.
2. Initialize balance to ₹5,000.
3. Read the number of transactions.
4. Repeat for each transaction.
5. Read transaction type and amount.
6. Check whether the amount is valid.
7. If deposit, add the amount to the balance.
8. If withdrawal, check balance and withdrawal limit.
9. Update balance if the withdrawal is valid.
10. Store valid transactions in a list.
11. Display transaction history and final balance.
12. Stop.

5. Flowchart





6. Pseudocode

```
START

balance = 5000
CREATE empty transaction list

INPUT n

FOR i = 1 TO n

  INPUT choice
  INPUT amount

  IF amount <= 0
    DISPLAY Invalid amount

  ELSE IF choice = 1
    balance = balance + amount
    STORE deposit transaction

  ELSE IF choice = 2

    IF amount <= balance AND amount <= 10000
      balance = balance - amount
      STORE withdrawal transaction

    ELSE
      DISPLAY Transaction rejected
    END IF
```

ELSE

DISPLAY Invalid choice

END IF

END FOR

DISPLAY transaction history

DISPLAY final balance

STOP

7. Python Program

```
balance = 5000
transactions = []

n = int(input("Enter number of transactions: "))

for i in range(n):

  print("\n1. Deposit")
  print("2. Withdraw")

  choice = int(input("Enter your choice: "))
  amount = float(input("Enter amount: "))

  if amount <= 0:
    print("Invalid amount")

  elif choice == 1:

    balance += amount

    transactions.append(("Deposit", amount))

    print("Amount Deposited Successfully")
```

```
  elif choice == 2:
    if amount <= balance and amount <= 10000:
      balance -= amount
      transactions.append(("Withdraw", amount))
      print("Amount Withdrawn Successfully")
    else:
      print("Transaction Rejected")
  else:
    print("Invalid Transaction Type")

print("\nATM TRANSACTION SUMMARY")

for transaction, amount in transactions:
  print(transaction, ": Rs.", amount)

print("\nFinal Balance: Rs.", balance)
```

8. Test Cases

Test	Transaction	Amount	Expected result
1	Deposit	₹2,000	Balance ₹7,000
2	Withdraw	₹1,000	Balance ₹6,000
3	Withdraw	₹8,000	Rejected if balance is ₹6,000
4	Deposit	-₹500	Invalid amount
5	Withdraw	₹12,000	Withdrawal limit exceeded

9. Sample Output

```
Enter number of transactions: 2
```

1. Deposit
2. Withdraw
Enter your choice: 1
Enter amount: 2000
Amount Deposited Successfully

1. Deposit
2. Withdraw
Enter your choice: 2
Enter amount: 1000
Amount Withdrawn Successfully

ATM TRANSACTION SUMMARY

Deposit : Rs. 2000.0
Withdraw : Rs. 1000.0

Final Balance: Rs. 6000.0

10. Conclusion

The ATM Transaction Simulation program performs deposits and withdrawals using conditional statements and loops. It maintains the account balance, validates transactions and displays the transaction history.

Case Study 5: Student Performance Trend Analysis

1. Problem Statement

Develop a Python program to compare student marks across multiple examinations. Calculate improvements and declines, identify consistently weak subjects and classify student performance.

2. Objectives

- Store marks of different subjects.
- Compare marks across examinations.
- Calculate improvement and decline.
- Identify weak subjects.
- Calculate the average and classify performance.

3. Assumptions

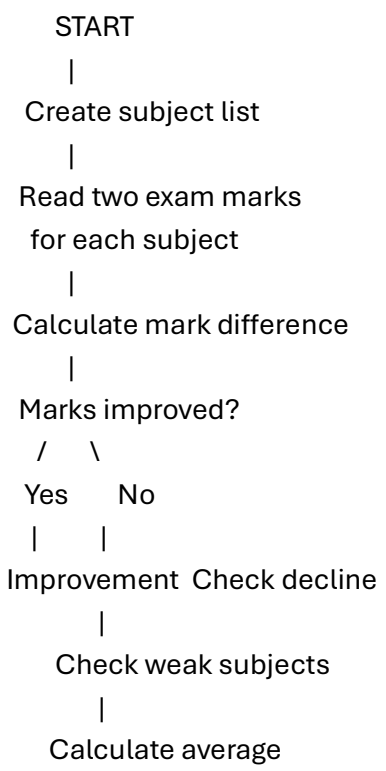
1. There are three subjects: Python, Mathematics and Physics.
2. Marks are entered for two examinations.

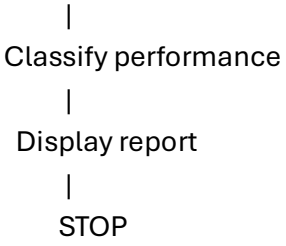
3. Maximum marks for each subject are 100.
4. Minimum pass mark is 35.
5. A subject is considered weak if marks are below 35 in both examinations.
6. Performance classification:
    - Average 75 or above: Excellent.
    - Average 60–74.99: Good.
    - Average 35–59.99: Average.
    - Below 35: Poor.

4. Algorithm

1. Start.
2. Create a list of subject names.
3. Create two empty lists for examination marks.
4. Read marks for each subject in both examinations.
5. Store marks in the respective lists.
6. Compare examination marks.
7. Calculate improvement or decline.
8. Identify consistently weak subjects.
9. Calculate the second examination average.
10. Classify student performance.
11. Display the complete report.
12. Stop.

5. Flowchart





6.Pseudocode

```
START

subjects = [Python, Maths, Physics]

CREATE exam1 list
CREATE exam2 list

FOR each subject

  INPUT first exam marks
  INPUT second exam marks

  STORE marks in lists

END FOR

FOR each subject

  difference = exam2 - exam1

  IF difference > 0
```

```
      DISPLAY Improvement

    ELSE IF difference < 0
      DISPLAY Decline

    ELSE
      DISPLAY No Change
    END IF

    IF exam1 < 35 AND exam2 < 35
      DISPLAY Consistently Weak Subject
    END IF

END FOR

average = SUM(exam2) / number of subjects

IF average >= 75
    DISPLAY Excellent

ELSE IF average >= 60
    DISPLAY Good

ELSE IF average >= 35
    DISPLAY Average

ELSE
    DISPLAY Poor
```

END IF

STOP

7. Python Program

```
subjects = ["Python", "Maths", "Physics"]

exam1 = []
exam2 = []

for subject in subjects:

  print("\nEnter marks for", subject)

  m1 = int(input("First Exam Marks: "))
  m2 = int(input("Second Exam Marks: "))

  if m1 < 0 or m1 > 100 or m2 < 0 or m2 > 100:
    print("Invalid marks")
    exit()

  exam1.append(m1)
  exam2.append(m2)

print("\nSTUDENT PERFORMANCE REPORT")

for i in range(len(subjects)):

  difference = exam2[i] - exam1[i]

  print("\nSubject:", subjects[i])
  print("First Exam:", exam1[i])
  print("Second Exam:", exam2[i])

  if difference > 0:
    print("Improvement:", difference)
```

```
    elif difference < 0:
        print("Decline:", abs(difference))

    else:
        print("No Change")

    if exam1[i] < 35 and exam2[i] < 35:
        print("Consistently Weak Subject")

average = sum(exam2) / len(subjects)

print("\nOverall Average:", round(average, 2))

if average >= 75:
    print("Performance: Excellent")

elif average >= 60:
    print("Performance: Good")

elif average >= 35:
    print("Performance: Average")

else:
    print("Performance: Poor")
```

8. Test Cases

Subject	**Exam 1**	**Exam 2**	**Expected result**
Python	60	80	Improvement
Maths	70	50	Decline
Physics	30	32	Consistently weak

9.Sample Output

Enter marks for Python

First Exam Marks: 60

Second Exam Marks: 80

Enter marks for Maths

First Exam Marks: 70

Second Exam Marks: 50

Enter marks for Physics

First Exam Marks: 30

Second Exam Marks: 32

STUDENT PERFORMANCE REPORT

Subject: Python

First Exam: 60

Second Exam: 80

Improvement: 20

Subject: Maths

First Exam: 70

Second Exam: 50

Decline: 20

Subject: Physics

First Exam: 30

Second Exam: 32

Improvement: 2

Consistently Weak Subject

Overall Average: 54.0

Performance: Average

10. Conclusion

The Student Performance Trend Analysis program compares examination marks and identifies improvements, declines and weak subjects. It helps students understand their academic performance.

Case Study 6: Password Policy Validation Engine

1. Problem Statement

Develop a Python program to validate passwords using predefined security rules. The program should check password length, uppercase letters, lowercase letters, digits, repeated characters and prohibited words. It should display all the conditions that are not satisfied.

2. Objectives

- Validate password length.
- Check uppercase and lowercase letters.
- Check the presence of digits and special characters.
- Detect repeated characters.
- Identify prohibited words.
- Display password validation errors.

3. Assumptions

The following sample password rules are assumed:

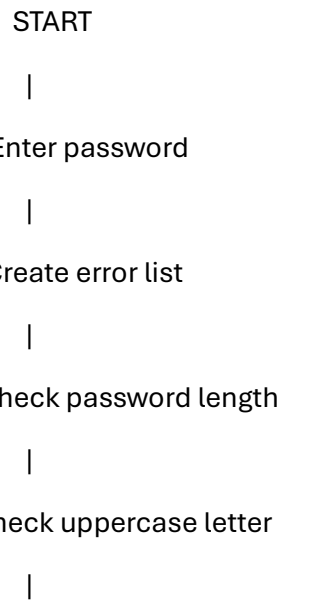
1. Minimum password length is 8 characters.
2. At least one uppercase letter is required.
3. At least one lowercase letter is required.
4. At least one digit is required.
5. At least one special character is required.
6. Three or more consecutive identical characters are not allowed.
7. Prohibited words: password, admin, 1234.

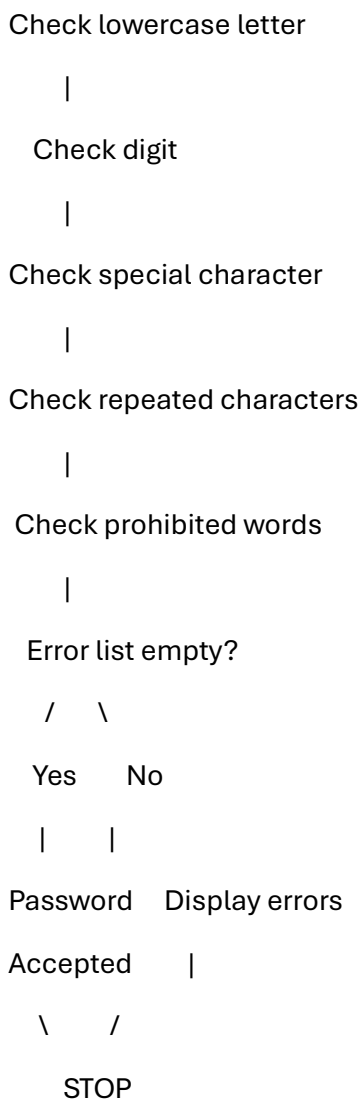
Note: These specific rules are assumptions for implementing the case study; the PDF does not provide exact password rules.

4. Algorithm

1. Start.
2. Read the password.
3. Create an empty error list.
4. Check minimum password length.
5. Check uppercase and lowercase letters.
6. Check whether a digit is present.
7. Check whether a special character is present.
8. Check consecutive repeated characters.
9. Check prohibited words.
10. Store all failed conditions in the error list.
11. If the error list is empty, accept the password.
12. Otherwise, display all errors.
13. Stop.

5. Flowchart





6. Pseudocode

START

INPUT password

CREATE empty errors list

IF length(password) < 8

ADD length error

```
IF no uppercase letter
    ADD uppercase error

IF no lowercase letter
    ADD lowercase error

IF no digit
    ADD digit error

IF no special character
    ADD special character error

CHECK consecutive repeated characters

IF three or more identical consecutive characters
    ADD repetition error

CHECK prohibited words

IF prohibited word found
    ADD prohibited word error

IF errors list is empty
    DISPLAY Password Accepted

ELSE
    DISPLAY Password Rejected
```

```
    FOR each error
        DISPLAY error
    END FOR

END IF

STOP
```

7.Python Program

```
password = input("Enter your password: ")

errors = []

if len(password) < 8:
    errors.append("Minimum 8 characters required")

if not any(ch.isupper() for ch in password):
    errors.append("At least one uppercase letter required")

if not any(ch.islower() for ch in password):
    errors.append("At least one lowercase letter required")

if not any(ch.isdigit() for ch in password):
    errors.append("At least one digit required")

if not any(not ch.isalnum() for ch in password):
```

```
    errors.append("At least one special character required")

# Check consecutive repeated characters
for i in range(len(password) - 2):

    if password[i] == password[i + 1] == password[i + 2]:
        errors.append("Three consecutive repeated characters not allowed")
        break

# Check prohibited words
prohibited = ["password", "admin", "1234"]

for word in prohibited:

    if word in password.lower():
        errors.append("Prohibited word found: " + word)

if len(errors) == 0:

    print("\nStrong Password")
    print("Password Accepted")

else:

    print("\nWeak Password")
    print("Password Rejected")

    print("\nErrors:")
```

```
    for error in errors:
        print("-", error)
```

8. Test Cases

Test	Password	Expected result
1	Hello@123	Accepted
2	hello123	Uppercase and special character errors
3	HELLO@123	Lowercase error
4	Abc@1111	Repeated characters error
5	Admin@1234	Prohibited word errors
6	Ab@12	Minimum length error

9. Sample Output

Enter your password: Hello@123

Strong Password

Password Accepted

10. Conclusion

The Password Policy Validation program checks password security using strings, loops, conditional statements and lists. It identifies errors and helps users create stronger passwords.

Case Study 7: Text Compression Using Character Runs

1. Problem Statement

Develop a Python program to compress a string by grouping consecutive repeated characters. The program should count repeated characters, generate an encoded string and compare the original and compressed string lengths to determine whether compression is beneficial.

2. Objectives

- Accept a string from the user.
- Identify consecutive repeated characters.
- Count the occurrences of each character.
- Generate a compressed string.
- Compare original and compressed lengths.

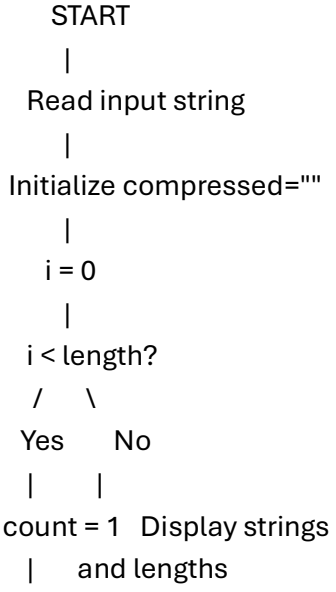
3. Assumptions

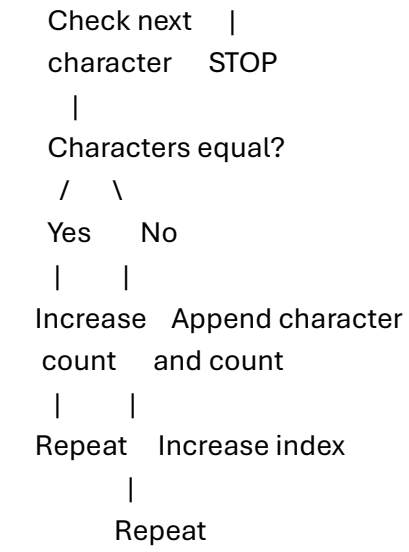
1. The input contains a string.
2. Consecutive repeated characters are grouped together.
3. Each character is followed by its occurrence count.
4. Single characters are also represented with count 1.
5. Compression is beneficial only when the compressed string is shorter.

4. Algorithm

1. Start.
2. Read a string from the user.
3. Initialize an empty compressed string.
4. Initialize index to zero.
5. Count consecutive repeated characters.
6. Append the character and its count to the compressed string.
7. Move to the next group of characters.
8. Repeat until the complete string is processed.
9. Calculate original and compressed lengths.
10. Compare both lengths.
11. Display the compression result.
12. Stop.

5. Flowchart





6. **Pseudocode**

```
START

INPUT text

compressed = ""
i = 0

WHILE i < length(text)

  count = 1

  WHILE next character is same

    count = count + 1
    i = i + 1

  END WHILE

  compressed = compressed + character + count

  i = i + 1

END WHILE

DISPLAY original string
DISPLAY compressed string
```

IF compressed length < original length

DISPLAY Compression is beneficial

ELSE

DISPLAY Compression is not beneficial

END IF

STOP

7. Python Program

```
text = input("Enter a string: ")

compressed = ""
i = 0

while i < len(text):

  count = 1

  while i + 1 < len(text) and text[i] == text[i + 1]:

    count += 1
    i += 1

  compressed += text[i] + str(count)

  i += 1

print("\nOriginal String:", text)
print("Compressed String:", compressed)

print("Original Length:", len(text))
print("Compressed Length:", len(compressed))

if len(compressed) < len(text):
  print("Compression is beneficial")
```

```
else:
  print("Compression is not beneficial")
```

8. Test Cases

Test	Input	Compressed output	Result
1	aaabbcccc	a3b2c4	Beneficial
2	abcd	a1b1c1d1	Not beneficial
3	aaaaaa	a6	Beneficial
4	aabb	a2b2	Not beneficial
5	xxxyyyzz	x3y3z2	Beneficial

9. Sample Output

```
Enter a string: aaabbcccc

Original String: aaabbcccc
Compressed String: a3b2c4

Original Length: 9
Compressed Length: 6

Compression is beneficial
```

10. Conclusion

The Text Compression program groups consecutive repeated characters and generates an encoded string. It compares the original and compressed lengths to determine whether compression reduces the string size.

Case Study 8: Student Marks Matrix Analysis

1. Problem Statement

Develop a Python program to analyse a matrix containing marks of multiple students across different subjects. Calculate student-wise totals and averages,

subject-wise averages, identify the top-performing student and detect subjects with low average marks.

2. Objectives

- Store student marks using nested lists.
- Calculate student-wise totals.
- Calculate student-wise averages.
- Calculate subject-wise averages.
- Identify the top-performing student.
- Detect subjects with low average performance.

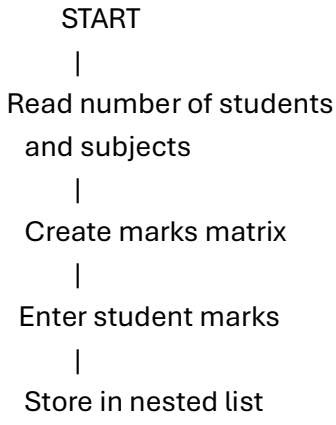
3. Assumptions

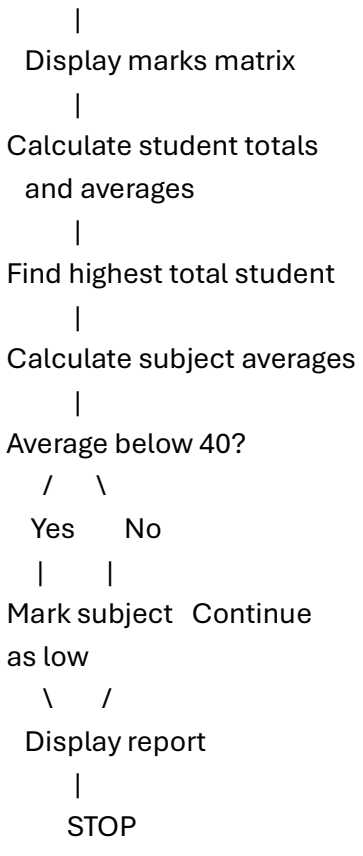
1. There are multiple students and subjects.
2. Marks are entered between 0 and 100.
3. All students have the same number of subjects.
4. A subject is considered low-performing if its average is below 40.
5. The student with the highest total marks is the topper.

4. Algorithm

1. Start.
2. Read the number of students and subjects.
3. Create an empty nested list.
4. Read marks for every student.
5. Store marks in the nested list.
6. Display the marks matrix.
7. Calculate each student's total and average.
8. Identify the student with the highest total.
9. Calculate each subject's average.
10. Identify subjects with averages below 40.
11. Display the analysis report.
12. Stop.

5. Flowchart





6. Pseudocode

```
START

INPUT students
INPUT subjects

CREATE empty marks matrix

FOR each student

  CREATE empty row

  FOR each subject

    INPUT mark
    ADD mark to row

  END FOR

  ADD row to matrix
```

END FOR

FOR each student

total = SUM of student marks
average = total / subjects

DISPLAY total and average

END FOR

FIND student with highest total

FOR each subject

total = 0

FOR each student

total = total + marks[student][subject]

END FOR

average = total / students

DISPLAY subject average

IF average < 40
DISPLAY Low Performance Subject
END IF

END FOR

DISPLAY topper

STOP

7. Python Program

```
students = int(input("Enter number of students: "))
```

```
subjects = int(input("Enter number of subjects: "))

if students <= 0 or subjects <= 0:

  print("Invalid number of students or subjects")

else:

  marks = []

  for i in range(students):

    row = []

    print("\nStudent", i + 1)

    for j in range(subjects):

      mark = int(input("Enter marks: "))

      if mark < 0 or mark > 100:
        print("Invalid marks")
        exit()

      row.append(mark)

    marks.append(row)

  print("\nSTUDENT MARKS MATRIX")

  for row in marks:
    print(row)

  totals = []

  print("\nSTUDENT-WISE ANALYSIS")

  for i in range(students):

    total = sum(marks[i])
    average = total / subjects
```

```
    totals.append(total)

    print("Student", i + 1)
    print("Total:", total)
    print("Average:", round(average, 2))

topper = totals.index(max(totals))

print("\nTopper: Student", topper + 1)
print("Highest Total:", max(totals))

print("\nSUBJECT-WISE ANALYSIS")

for j in range(subjects):

    total = 0

    for i in range(students):
        total += marks[i][j]

    average = total / students

    print("Subject", j + 1)
    print("Average:", round(average, 2))

    if average < 40:
        print("Low Performance Subject")
```

8. Test Cases

Consider the following marks matrix:

Student	**Python**	**Maths**	**Physics**
Student 1	80	70	90
Student 2	60	50	40
Student 3	30	35	25

Expected calculations:

Student	Total	Average
Student 1	240	80
Student 2	150	50
Student 3	90	30

Subject averages:

- Python = 56.67
- Maths = 51.67
- Physics = 51.67

Topper: Student 1.

No subject has an average below 40 in this test.

9.Sample Output

STUDENT MARKS MATRIX

[80, 70, 90]

[60, 50, 40]

[30, 35, 25]

STUDENT-WISE ANALYSIS

Student 1

Total: 240

Average: 80.0

Student 2

Total: 150

Average: 50.0

Student 3

Total: 90

Average: 30.0

Topper: Student 1

Highest Total: 240

SUBJECT-WISE ANALYSIS

Subject 1

Average: 56.67

Subject 2

Average: 51.67

Subject 3

Average: 51.67

10. Conclusion

The Student Marks Matrix Analysis program uses nested lists to store and analyse student marks. It calculates totals, averages, identifies the topper and detects low-performing subjects.

Case Study 9: Tuple-Based Product Catalogue Audit

1. Problem Statement

Develop a Python program to maintain a product catalogue using tuples. Each tuple contains a product identifier, price and stock quantity. The program should detect invalid records, calculate total inventory value and identify products that require replenishment.

2. Objectives

- Store product details using tuples.

- Access tuple elements through unpacking.
- Validate product prices and stock quantities.
- Calculate product-wise inventory values.
- Calculate total inventory value.
- Identify products requiring replenishment.

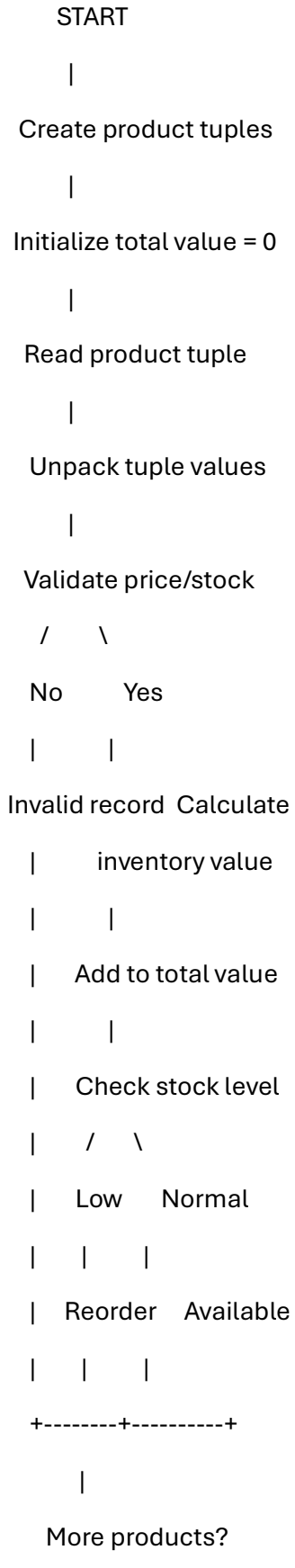
3. Assumptions

1. Each product contains a product name, price and stock quantity.
2. Product details are stored in tuples.
3. Price must not be negative.
4. Stock quantity must not be negative.
5. Products with stock quantity of 3 or less require replenishment.
6. The product catalogue contains sample records.

4. Algorithm

1. Start.
2. Create a list of product tuples.
3. Initialize total inventory value to zero.
4. Read each product tuple.
5. Unpack product name, price and stock.
6. Validate price and stock.
7. Calculate inventory value by multiplying price and stock.
8. Add inventory value to the total.
9. Check whether stock is zero.
10. Check whether stock is below or equal to the replenishment limit.
11. Display product details and warnings.
12. Display total inventory value.
13. Stop.

5. Flowchart


START
Create product tuples
Initialize total value = 0
Read product tuple
Unpack tuple values
Validate price/stock
No
Yes
Invalid record
Calculate inventory value
Add to total value
Check stock level
Low
Normal
Reorder
Available
More products?

/ \

Yes No

| |

Repeat Display total

|

STOP

6.Pseudocode

```
START

CREATE product tuples

total_value = 0

FOR each product

  UNPACK name, price, stock

  IF price < 0 OR stock < 0

    DISPLAY Invalid record

  ELSE

    value = price * stock

    total_value = total_value + value

    DISPLAY product details
    DISPLAY inventory value

    IF stock = 0
      DISPLAY Out of Stock

    ELSE IF stock <= 3
      DISPLAY Replenishment Required
```

```
        ELSE
          DISPLAY Stock Available

        END IF

      END IF

    END FOR

    DISPLAY total inventory value

    STOP
```

7. Python Program

```
products = [
  ("Laptop", 50000, 5),
  ("Mobile", 20000, 10),
  ("Keyboard", 800, 0),
  ("Mouse", 500, 3),
  ("Headphones", 1500, 2),
  ("Invalid Product", -100, 5)
]

total_value = 0

print("PRODUCT CATALOGUE AUDIT")

for product in products:

  name, price, stock = product

  print("\nProduct:", name)

  if price < 0 or stock < 0:

    print("Invalid Product Record")

    continue

  value = price * stock
```

```
    total_value += value

    print("Price: Rs.", price)
    print("Stock:", stock)
    print("Inventory Value: Rs.", value)

    if stock == 0:

        print("Out of Stock")

    elif stock <= 3:

        print("Low Stock - Reorder Required")

    else:

        print("Stock Available")

print("\nTotal Inventory Value: Rs.", total_value)
```

8. Test Cases

Product	Price	Stock	Expected result
Laptop	₹50,000	5	Stock available
Mobile	₹20,000	10	Stock available
Keyboard	₹800	0	Out of stock
Mouse	₹500	3	Reorder required
Headphones	₹1,500	2	Reorder required
Invalid Product	-₹100	5	Invalid record

10. Conclusion

The Tuple-Based Product Catalogue Audit program stores product information using tuples. It uses tuple unpacking to access product details, validates records, calculates inventory values and identifies products that require replenishment.

Case Study 10: Food Waste Monitoring System

1. Problem Statement

Develop a Python program to monitor food waste over several days. The program should compare the number of meals prepared, meals served and leftover meals. It should calculate the food waste percentage and identify days when food waste exceeds the acceptable limit.

2. Objectives

- Store daily food preparation details.
- Calculate leftover meals.
- Calculate daily food waste percentage.
- Identify days with excessive food waste.
- Calculate total weekly food waste.
- Generate a weekly food waste report.

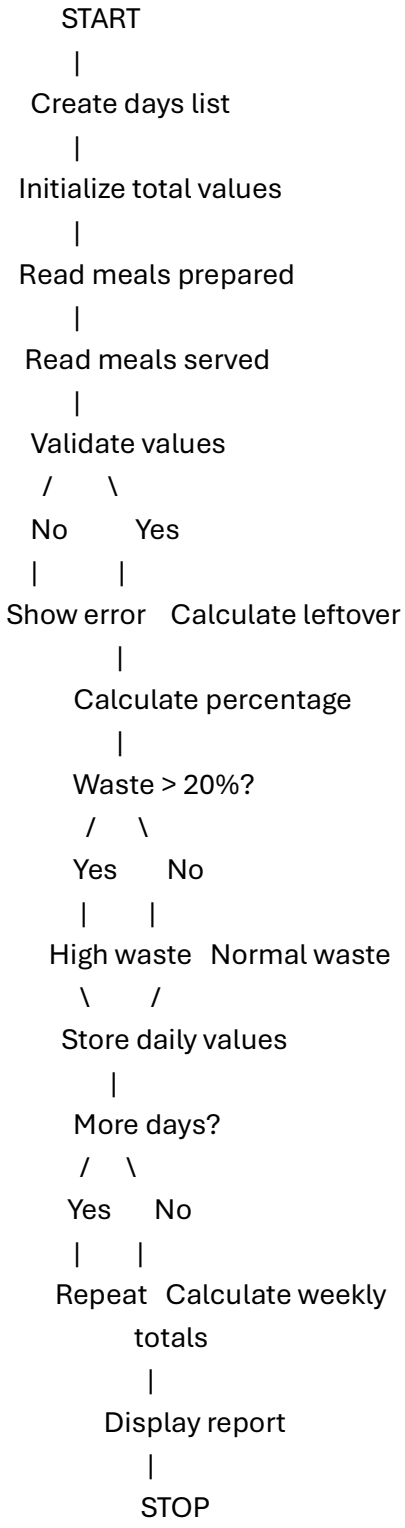
3. Assumptions

1. Food waste is monitored for five days.
2. Meals prepared and served are entered by the user.
3. Leftover meals = Prepared meals − Served meals.
4. Food waste percentage is calculated based on meals prepared.
5. Food waste above 20% is considered excessive.
6. Negative values and served meals greater than prepared meals are invalid.

4. Algorithm

1. Start.
2. Create a list containing five working days.
3. Initialize total prepared meals and total waste to zero.
4. Read the number of meals prepared.
5. Read the number of meals served.
6. Validate the entered values.
7. Calculate leftover meals.
8. Calculate food waste percentage.
9. Check whether waste percentage exceeds 20%.
10. Store daily results.
11. Repeat for all five days.
12. Calculate weekly total meals and total waste.
13. Display the weekly food waste report.
14. Stop.

5. Flowchart



6. Pseudocode

START

```
days = [Monday, Tuesday, Wednesday, Thursday, Friday]

total_prepared = 0
total_waste = 0

FOR each day

  INPUT prepared
  INPUT served

  IF prepared < 0 OR served < 0 OR served > prepared

    DISPLAY Invalid input

  ELSE

    leftover = prepared - served

    IF prepared > 0

      percentage = leftover / prepared * 100

    ELSE

      percentage = 0

    END IF

    total_prepared = total_prepared + prepared

    total_waste = total_waste + leftover

    DISPLAY leftover and percentage

    IF percentage > 20

      DISPLAY High Food Waste

    ELSE

      DISPLAY Food Waste Under Control
```

```
    END IF

  END IF

END FOR

DISPLAY total prepared meals
DISPLAY total wasted meals

IF total_prepared > 0

  weekly_percentage = total_waste / total_prepared * 100

  DISPLAY weekly_percentage

END IF

STOP
```

7. Python Program

```
days = ["Monday", "Tuesday", "Wednesday",
    "Thursday", "Friday"]

prepared = []
served = []
leftover = []

total_waste = 0
total_prepared = 0

for day in days:

  print("\n", day)

  p = int(input("Enter meals prepared: "))
  s = int(input("Enter meals served: "))

  if p < 0 or s < 0 or s > p:

    print("Invalid input")
```

```
        continue

    waste = p - s

    prepared.append(p)
    served.append(s)
    leftover.append(waste)

    total_waste += waste
    total_prepared += p

    if p > 0:
        percentage = (waste / p) * 100
    else:
        percentage = 0

    print("Leftover Meals:", waste)

    print("Waste Percentage:",
          round(percentage, 2), "%")

    if percentage > 20:

        print("High Food Waste Detected")

    else:

        print("Food Waste Under Control")

print("\nWEEKLY FOOD WASTE REPORT")

print("Total Meals Prepared:", total_prepared)
print("Total Meals Served:", sum(served))
print("Total Meals Wasted:", total_waste)

if total_prepared > 0:

    waste_percentage = (total_waste / total_prepared) * 100

    print("Overall Waste Percentage:",
          round(waste_percentage, 2), "%")
```

```
else:

  print("No meals prepared")
```

8. Test Cases

Day	Prepared	Served	Leftover	Waste %
Monday	100	90	10	10%
Tuesday	120	90	30	25%
Wednesday	150	120	30	20%
Thursday	100	70	30	30%
Friday	80	75	5	6.25%

Weekly calculations:

- Total meals prepared = 550
- Total meals served = 445
- Total meals wasted = 105
- Overall waste percentage = 19.09%

9. Sample Output

```
Monday
Enter meals prepared: 100
Enter meals served: 90
Leftover Meals: 10
Waste Percentage: 10.0 %
Food Waste Under Control

Tuesday
Enter meals prepared: 120
Enter meals served: 90
Leftover Meals: 30
Waste Percentage: 25.0 %
High Food Waste Detected

Wednesday
Enter meals prepared: 150
Enter meals served: 120
Leftover Meals: 30
Waste Percentage: 20.0 %
Food Waste Under Control
```

Thursday
Enter meals prepared: 100
Enter meals served: 70
Leftover Meals: 30
Waste Percentage: 30.0 %
High Food Waste Detected

Friday
Enter meals prepared: 80
Enter meals served: 75
Leftover Meals: 5
Waste Percentage: 6.25 %
Food Waste Under Control

WEEKLY FOOD WASTE REPORT

Total Meals Prepared: 550
Total Meals Served: 445
Total Meals Wasted: 105
Overall Waste Percentage: 19.09 %

10. Conclusion

The Food Waste Monitoring program calculates daily and weekly food waste using lists, loops and conditional statements. It identifies days with excessive food waste and helps improve food management.